PixelKit

App de escritorio multiplataforma para procesamiento de imagenes

---

Python 3.11+ | CustomTkinter | Pillow | rembg | Windows / macOS / Linux

Tabla de contenidos

1. Vision general del proyecto
2. Stack tecnologico y decisiones de diseno
3. Arquitectura y estructura de carpetas
4. Modulos: especificacion y codigo
   4.1 Comprimir
   4.2 Convertir formato
   4.3 Quitar fondo
   4.4 Redimensionar y recortar
   4.5 Renombrar en lote
   4.6 Extractor de paleta de colores
   4.7 LQIP + Base64
   4.8 Metadatos EXIF
   4.9 Watermark batch
   4.10 Smart Optimizer
5. Interfaz de usuario (UI con CustomTkinter)
   5.1 Layout principal
   5.2 Componentes reutilizables
   5.3 Guia de referencia UI para Google Stitch
   5.4 Codigo base de ventana principal y sidebar
   5.5 Drop Zone con drag & drop
6. Dependencias y configuracion del entorno
7. Plan de desarrollo por fases
8. Build y distribucion con PyInstaller

1. Vision general del proyecto

PixelKit es una aplicacion de escritorio multiplataforma (Windows, macOS, Linux) construida en Python, disenada para reemplazar las herramientas web de procesamiento de imagenes que limitan el uso con planes de pago, restricciones de tamano y subida de datos a servidores externos.

El objetivo es tener una app rapida, sin internet, sin limites, con procesamiento local 100%.

Problema que resuelve

Las apps web como Squoosh, Remove.bg, TinyPNG, Convertio tienen en comun:
- Limites de archivos por dia / mes en el plan gratuito
- Subis tus imagenes a servidores de terceros (privacidad)
- Lentitud por upload/download de archivos
- Sin acceso offline
- Funciones que queres no estan en la misma app

Solucion PixelKit

- Procesamiento 100% local, sin internet
- Sin limites de cantidad ni tamano
- Todo en un solo lugar: comprimir, convertir, quitar fondo, paleta...
- Modo oscuro nativo, interfaz limpia
- Drag & drop, batch processing, previews en tiempo real
- Exportable como .exe / .app para compartir con cualquiera

2. Stack tecnologico y decisiones de diseno

GUI Framework: CustomTkinter

CustomTkinter es la opcion correcta para este proyecto. Es Tkinter con modo oscuro nativo, widgets modernos, y se ve bien en los 3 sistemas operativos sin configuracion extra. Alternativas descartadas:

- PyQt6 / PySide6: mas potente pero mas complejo, licencia comercial en Qt6, curva de aprendizaje mayor
- Kivy: orientado a touchscreen/mobile, no se ve como app desktop nativa
- wxPython: interfaz mas nativa pero menos atractiva visualmente, mas dificil de customizar
- Tkinter vanilla: demasiado basico, sin modo oscuro, widgets feos por defecto

Procesamiento: Pillow

Pillow cubre el 85% de todas las operaciones necesarias: comprimir, convertir, redimensionar, recortar, watermark, LQIP, paleta (via getcolors), metadatos basicos. Es la libreria estandar de Python para imagenes, estable, bien documentada y rapida.

Quitar fondo: rembg

rembg usa el modelo U2Net de deep learning para segmentacion de imagen. El resultado es significativamente mejor que cualquier enfoque clasico con OpenCV. El modelo (~170MB) se descarga automaticamente en el primer uso y queda cacheado localmente. Requiere onnxruntime como backend de inferencia.

Tabla de dependencias

Libreria	Version	Uso
customtkinter	>=5.2.0	GUI moderna, modo oscuro, CTk widgets
Pillow	>=10.0.0	Procesamiento core: resize, crop, compress, convert
rembg	>=2.0.50	Quitar fondo con IA (modelo U2Net)
onnxruntime	>=1.16.0	Runtime para el modelo de rembg
colorthief	>=0.2.1	Extraccion de paleta de colores dominantes
piexif	>=1.1.3	Leer y modificar metadatos EXIF
pillow-avif-plugin	>=1.4.0	Soporte para formato AVIF
tkinterdnd2	>=0.3.0	Drag & drop nativo multiplataforma
pyinstaller	>=6.0.0	Empaquetar la app en .exe / .app / binario

3. Arquitectura y estructura de carpetas

La app sigue una arquitectura modular simple: un nucleo de procesamiento (core/), modulos independientes por funcionalidad (modules/), y una capa UI completamente separada (ui/). Esto permite agregar modulos nuevos sin tocar el codigo existente.

```
pixelkit/
  main.py                 # Entry point
  app.py                  # App class, ventana principal
  requirements.txt
  build.spec              # PyInstaller config

  core/
    __init__.py
    image_processor.py    # Procesamiento base (Pillow)
    file_utils.py         # IO, drag & drop, paths
    settings.py           # Config persistente (JSON)

  modules/
    __init__.py
    compress.py           # Compresion y optimizacion
    convert.py            # Conversion de formatos
    remove_bg.py          # Quitar fondo (rembg)
    resize_crop.py        # Redimensionar y recortar
    rename_batch.py       # Renombrar en lote
    palette.py            # Extractor de paleta
    watermark.py          # Marca de agua batch
    metadata.py           # EXIF / metadatos
    lqip.py               # Low Quality Image Placeholder
    smart_optimizer.py    # Sugeridor de formato optimo

  ui/
    __init__.py
    main_window.py        # CTk ventana principal
    sidebar.py            # Navegacion lateral
    drop_zone.py          # Zona drag & drop
    preview_panel.py      # Panel de preview
    frames/               # Una frame por modulo
      compress_frame.py
      convert_frame.py
      remove_bg_frame.py
      resize_frame.py
      rename_frame.py
      palette_frame.py
      watermark_frame.py
      metadata_frame.py
      lqip_frame.py
      optimizer_frame.py

  assets/
    icon.ico
    icon.png
    logo.png
```

Principios de diseno del codigo:

- Cada modulo en modules/ es independiente: recibe paths, retorna resultados, sin dependencias de UI

- La UI en ui/frames/ solo llama funciones de modules/, nunca procesa imagenes directamente
- core/settings.py maneja configuracion persistente (ultimo directorio, calidad default, tema)
- core/file_utils.py centraliza toda la logica de paths, nombres, directorios de salida

4. Modulos: especificacion y codigo

Tabla resumen de modulos

Modulo	Deps	Prioridad	Notas
Comprimir	Pillow	Alta	Core - uso diario
Convertir formato	Pillow	Alta	Core - uso diario
Quitar fondo	rembg + onnxruntime	Media	Lento en primera ejecucion (descarga modelo)
Redimensionar / Recortar	Pillow	Alta	Core - uso diario
Renombrar en lote	pathlib (stdlib)	Alta	Muy pedido, sin dependencias extra
Extractor de paleta	colorthief	Media	Diferenciador - export CSS/HEX
LQIP + Base64	Pillow (stdlib)	Alta	Unico en apps desktop, muy util en web
Metadatos EXIF	piexif + Pillow	Media	Util para privacidad y organizacion
Watermark batch	Pillow	Media	Texto o imagen sobre lote completo
Smart Optimizer	Pillow (logica propia)	Alta	Recomienda mejor formato + estima ahorro

4.1 Modulo: Comprimir

Funcionalidad: Reduce el tamano de archivo de una imagen controlando la calidad. Soporta JPG, WebP, PNG y AVIF. Incluye preview del tamano estimado en tiempo real antes de guardar.

Parametros de entrada: ruta de imagen, calidad (0-100), formato de salida, directorio de salida, mantener/quitar EXIF.

Retorno: diccionario con tamano original, tamano comprimido, porcentaje de reduccion.

```
# modules/compress.py
from PIL import Image
from pathlib import Path

FORMATS_LOSSY = ['JPEG', 'WEBP', 'AVIF']
FORMATS_LOSSLESS = ['PNG', 'WEBP', 'GIF', 'BMP']

def compress_image(input_path: str, output_path: str,
                   quality: int = 85,
                   format: str = None,
                   keep_metadata: bool = False) -> dict:
    '''
    Comprime una imagen.
    Retorna dict con original_size, compressed_size, ratio.
    '''
    img = Image.open(input_path)
    original_size = Path(input_path).stat().st_size
```

```
    fmt = format or img.format or 'JPEG'
    fmt = fmt.upper()

    save_kwargs = {}
    if fmt == 'JPEG':
        img = img.convert('RGB')
        save_kwargs = {'quality': quality, 'optimize': True}
    elif fmt == 'WEBP':
        save_kwargs = {'quality': quality, 'method': 6}
    elif fmt == 'PNG':
        save_kwargs = {'optimize': True}

    if not keep_metadata and fmt in ('JPEG', 'WEBP'):
        # Quita EXIF por defecto
        if hasattr(img, '_getexif'):
            img.info.pop('exif', None)

    img.save(output_path, fmt, **save_kwargs)
    compressed_size = Path(output_path).stat().st_size

    return {
        'original_size': original_size,
        'compressed_size': compressed_size,
        'ratio': round((1 - compressed_size / original_size) * 100, 1),
        'format': fmt
    }

def estimate_size(input_path: str, quality: int,
                  format: str = 'WEBP') -> int:
    '''Estima tamano sin guardar al disco.'''
    import io
    img = Image.open(input_path)
    buffer = io.BytesIO()
    img.save(buffer, format=format, quality=quality)
    return buffer.tell()
```

4.2 Modulo: Convertir formato

Funcionalidad: Convierte entre JPG, PNG, WebP, GIF, BMP, TIFF, ICO, AVIF. Maneja automaticamente el caso de transparencia al convertir a JPEG (rellena con blanco). Soporta procesamiento en lote con callback de progreso.

```
# modules/convert.py
from PIL import Image
from pathlib import Path

SUPPORTED = {
    '.jpg': 'JPEG', '.jpeg': 'JPEG', '.png': 'PNG',
    '.webp': 'WEBP', '.gif': 'GIF', '.bmp': 'BMP',
    '.tiff': 'TIFF', '.ico': 'ICO',
}

def convert_image(input_path: str, target_format: str,
                  output_dir: str = None) -> str:
    '''
    Convierte una imagen a otro formato.
    Retorna el path del archivo resultante.
    '''
    img = Image.open(input_path)
    src = Path(input_path)
```

```
    fmt = target_format.upper()

    # Manejo de transparencia al convertir a JPEG
    if fmt == 'JPEG' and img.mode in ('RGBA', 'LA', 'P'):
        background = Image.new('RGB', img.size, (255, 255, 255))
        if img.mode == 'P':
            img = img.convert('RGBA')
        background.paste(img, mask=img.split()[-1]
                         if img.mode == 'RGBA' else None)
        img = background

    ext = '.' + fmt.lower()
    if fmt == 'JPEG': ext = '.jpg'
    out_dir = Path(output_dir) if output_dir else src.parent
    out_path = out_dir / (src.stem + ext)

    img.save(str(out_path), fmt)
    return str(out_path)

def batch_convert(paths: list, target_format: str,
                  output_dir: str = None,
                  progress_cb=None) -> list:
    results = []
    for i, path in enumerate(paths):
        out = convert_image(path, target_format, output_dir)
        results.append(out)
        if progress_cb:
            progress_cb(i + 1, len(paths))
    return results
```

4.3 Modulo: Quitar fondo (IA)

Funcionalidad: Usa el modelo U2Net via rembg para eliminar el fondo de la imagen. Permite elegir entre fondo transparente (PNG) o rellenar con un color solido (JPEG/PNG). Importante: el primer uso descarga el modelo (~170MB).

```
# modules/remove_bg.py
# Requiere: pip install rembg onnxruntime
from rembg import remove as rembg_remove
from PIL import Image
from pathlib import Path

def remove_background(input_path: str,
                      output_path: str = None,
                      bg_color: tuple = None) -> str:
    '''
    Quita el fondo de una imagen usando IA.
    bg_color: None = fondo transparente,
              (R,G,B) = rellena con color solido.
    '''
    img = Image.open(input_path)
    result = rembg_remove(img)  # RGBA con fondo transparente

    if bg_color:
        background = Image.new('RGBA', result.size, (*bg_color, 255))
        background.paste(result, mask=result.split()[3])
        result = background.convert('RGB')
        fmt, ext = 'JPEG', '.jpg'
    else:
        fmt, ext = 'PNG', '.png'
```

```
    if not output_path:
        src = Path(input_path)
        output_path = str(src.parent / (src.stem + '_nobg' + ext))

    result.save(output_path, fmt)
    return output_path

# NOTA: el primer uso descarga el modelo (~170MB).
# Despues queda cacheado en ~/.u2net/
```

Nota importante sobre rembg

El modelo U2Net se descarga automaticamente en ~/.u2net/ en el primer uso.
Tiempo de descarga: ~1-3 minutos segun conexion.
Luego del primer uso queda cacheado y funciona offline.
Para el .exe final, se puede pre-empaquetar el modelo en el build de PyInstaller.

4.4 Modulo: Redimensionar y recortar

Funcionalidad: Redimensiona por pixeles exactos, porcentaje o presets predefinidos (1080p, 4K, Instagram 1:1, Story 9:16, Twitter header, etc.). Mantiene aspect ratio o lo fuerza. El recorte soporta seleccion libre y ratios fijos.

- Redimensionar: Image.resize(target, Image.LANCZOS) para calidad maxima
- Recortar: Image.crop((left, top, right, bottom))
- Presets: diccionario con resolutions populares, expandible
- Thumbnail inteligente: Image.thumbnail() mantiene aspecto automaticamente

4.5 Modulo: Renombrar en lote

Funcionalidad: Selecciona N imagenes, define un prefijo, y las renombra con numeracion secuencial. Preview de nombres antes de aplicar. Soporte para orden customizable via drag & drop en la lista.

- Ejemplo: 'foto_001.jpg', 'foto_002.jpg', 'foto_003.jpg'
- Padding configurable: 001, 01, 1
- Preserva extension original o fuerza conversion
- Vista previa: tabla con nombre original -> nombre nuevo antes de confirmar
- Operacion segura: copia en nuevo directorio o renombra in-place con confirmacion

4.6 Modulo: Extractor de paleta de colores

Funcionalidad: Extrae los N colores dominantes de una imagen. Exporta en HEX, CSS variables, JSON o formato Figma. Incluye copia al portapapeles.

```
# modules/palette.py
from colorthief import ColorThief
from PIL import Image

def extract_palette(image_path: str, n_colors: int = 6) -> list:
```

```
    '''
    Extrae N colores dominantes de la imagen.
    Retorna lista de (R, G, B) tuples.
    '''
    ct = ColorThief(image_path)
    if n_colors == 1:
        return [ct.get_color(quality=1)]
    return ct.get_palette(color_count=n_colors, quality=1)

def rgb_to_hex(rgb: tuple) -> str:
    return '#{:02X}{:02X}{:02X}'.format(*rgb)

def export_css_vars(colors: list, prefix: str = 'color') -> str:
    '''Genera variables CSS listas para copiar.'''
    lines = [':root {']
    for i, rgb in enumerate(colors):
        hex_val = rgb_to_hex(rgb)
        lines.append(f'  --{prefix}-{i+1}: {hex_val};')
    lines.append('}')
    return ' '.join(lines)

def export_for_figma(colors: list) -> str:
    '''Retorna lista de HEX separados por coma para Figma.'''
    return ', '.join(rgb_to_hex(c) for c in colors)
```

4.7 Modulo: LQIP + Base64 (Low Quality Image Placeholder)

Funcionalidad diferenciadora: Genera una version tiny y borrosa de la imagen en base64, lista para usar como placeholder de lazy loading en proyectos web. Esta tecnica es la que usan Next.js, Gatsby y otros frameworks modernos.

Caso de uso tipico: el desarrollador web tiene una imagen de 2MB. Genera un LQIP de 400 bytes en base64, lo pone como src inicial del <img>, y cuando carga la imagen real reemplaza el src. El usuario ve un blur instantaneo en lugar de espacio en blanco.

```
# modules/lqip.py
import base64, io
from PIL import Image, ImageFilter

def generate_lqip(input_path: str,
                  target_width: int = 20,
                  blur_radius: float = 1.0) -> dict:
    '''
    Genera un Low Quality Image Placeholder.
    Retorna dict con base64 string y data URI listo para usar.
    '''
    img = Image.open(input_path)
    ratio = img.height / img.width
    target_height = max(1, int(target_width * ratio))

    # Miniatura borrosa
    tiny = img.resize((target_width, target_height),
                      Image.LANCZOS)
    tiny = tiny.filter(ImageFilter.GaussianBlur(blur_radius))

    # Convertir a base64
    buffer = io.BytesIO()
    tiny.save(buffer, format='JPEG', quality=40)
    b64 = base64.b64encode(buffer.getvalue()).decode()
    data_uri = f'{b64}'
```

```
    return {
        'base64': b64,
        'data_uri': data_uri,
        'img_tag': f'<img src="{data_uri}" alt="placeholder" />',
        'bytes': len(buffer.getvalue()),
        'dimensions': (target_width, target_height)
    }
```

4.8 Modulo: Metadatos EXIF

Funcionalidad: Lee y muestra metadatos EXIF de imagenes JPG/TIFF: camara, lente, ISO, velocidad de obturacion, apertura, fecha, coordenadas GPS. Permite borrar todos los metadatos (util para privacidad antes de publicar en redes).

- Leer: piexif.load(path) retorna dict con todos los tags
- Borrar: guardar imagen sin datos EXIF via Pillow
- Export: guardar resumen como .txt
- GPS: convertir coordenadas DMS a decimal para mostrar en texto

4.9 Modulo: Watermark batch

Funcionalidad: Aplica una marca de agua de texto o imagen a una o mas imagenes. Configurable: posicion (9 puntos), opacidad, tamano de fuente, color.

- Texto: usa Pillow ImageDraw + ImageFont (requiere fuente .ttf)
- Imagen: pega el logo con transparencia usando Image.paste con mascara alpha
- Posicion: esquinas (TL, TR, BL, BR), centro, o coordenadas custom
- Batch: mismas opciones aplicadas a lista de imagenes con progress bar

4.10 Modulo: Smart Optimizer

Funcionalidad diferenciadora: Analiza una imagen y recomienda el formato optimo segun sus caracteristicas (foto vs grafico, presencia de transparencia, cantidad de colores). Estima el tamano en cada formato antes de convertir.

```
# modules/smart_optimizer.py
from PIL import Image
from pathlib import Path
import io

def analyze_image(path: str) -> dict:
    img = Image.open(path)
    has_alpha = img.mode in ('RGBA', 'LA') or (
        img.mode == 'P' and 'transparency' in img.info)

    # Detectar si es foto o grafico
    colors = img.convert('RGB').getcolors(maxcolors=10000)
    is_graphic = colors is not None and len(colors) < 256

    size_kb = Path(path).stat().st_size / 1024

    # Estimar tamanos en distintos formatos
    estimates = {}
```

```
    for fmt, q in [('WEBP',80),('JPEG',85),('PNG',None),('AVIF',75)]:
        buf = io.BytesIO()
        try:
            tmp = img.convert('RGB') if fmt == 'JPEG' else img
            if q: tmp.save(buf, fmt, quality=q)
            else: tmp.save(buf, fmt, optimize=True)
            estimates[fmt] = round(buf.tell() / 1024, 1)
        except: pass

    # Logica de recomendacion
    if has_alpha and is_graphic:
        rec = 'PNG'; reason = 'Grafico con transparencia'
    elif has_alpha:
        rec = 'WEBP'; reason = 'Foto/imagen con transparencia'
    elif is_graphic:
        rec = 'PNG'; reason = 'Grafico de colores planos'
    else:
        best = min(estimates, key=estimates.get)
        rec = best; reason = 'Mejor compresion para esta foto'

    savings = round((1 - estimates.get(rec, size_kb) / size_kb) * 100, 1)

    return {
        'recommended_format': rec,
        'reason': reason,
        'has_alpha': has_alpha,
        'is_graphic': is_graphic,
        'original_kb': round(size_kb, 1),
        'estimates_kb': estimates,
        'estimated_savings_pct': max(0, savings)
    }
```

5. Interfaz de usuario (CustomTkinter)

5.1 Layout principal

La app usa un layout de dos columnas: sidebar fijo de 200px a la izquierda, y area de contenido que ocupa el resto. Cada modulo es una CTkFrame que se trae al frente con tkraise(). No hay navegacion con tabs ni ventanas separadas.

Dimensiones y colores de la UI

Ventana: 1200 x 750px, minimo 900 x 600
Sidebar: 200px ancho, color fondo oscuro (#1a1a2e)
Boton activo en sidebar: azul (#2563eb / #1d4ed8)
Tipografia: CTkFont size=13 para botones, 14 para labels, 12 para subtitulos
Espaciado interno de frames: padx=20, pady=15
Drop zone: borde punteado azul, altura 160px, corner_radius=12
Barra de progreso: CTkProgressBar, visible durante batch processing

5.2 Componentes reutilizables

- DropZone: acepta drag & drop y click para browse, llama callback on_drop(paths)
- PreviewPanel: muestra imagen original vs resultado con slider A/B
- ProgressBar: barra + label de progreso, muestra 'N/M imagenes procesadas'
- FileList: lista scrolleable de imagenes cargadas con miniaturas
- ExportButton: boton principal de accion, verde, siempre visible abajo derecha
- InfoLabel: muestra tamano de archivo, dimensiones, formato de forma compacta

5.3 Guia de referencia UI para Google Stitch

Cuando vayas a disenar la UI en Google Stitch, usa esta especificacion como referencia para que el diseno sea fidedigno a lo que vas a construir con CTk:

Elemento	Stitch (descripcion)	CTk equivalente
Sidebar	Panel vertical izquierdo 200px, fondo oscuro profundo, lista de items con hover state	CTkFrame + CTkButton fg_color=transparent
Logo/titulo	Texto "PixelKit" bold 22px, padding top 20px, centrado	CTkLabel font=CTkFont(22, bold)
Item activo sidebar	Relleno azul (#2563eb), texto blanco, corner radius 6	CTkButton configure(fg_color="#2563eb")
Area contenido	Fondo ligeramente diferente al sidebar, padding 20px	CTkFrame corner_radius=0
Drop zone	Rectangulo de linea punteada azul, texto centrado, 160px alto, click y drag	CTkFrame border_width=2 border_color=blue

Slider de calidad	Slider horizontal 0-100, label que muestra el valor en tiempo real	CTkSlider + CTkLabel
Progress bar	Barra lineal azul, label "N/M imagenes", visible solo durante proceso	CTkProgressBar
Boton exportar	Boton verde prominente, full-width o al pie derecho, texto "Exportar"	CTkButton fg_color=green
Preview A/B	Dos imagenes lado a lado con divisor arrastrable	Canvas con linea divisora custom
Swatches paleta	Rectangulos de color con hex debajo, 6 en fila	CTkFrame con Canvas para colores

5.4 Codigo base: main.py, app.py, main_window.py, sidebar.py

```
# main.py
import customtkinter as ctk
from app import PixelKitApp

ctk.set_appearance_mode('dark')  # 'dark' | 'light' | 'system'
ctk.set_default_color_theme('blue')

if __name__ == '__main__':
    app = PixelKitApp()
    app.mainloop()
```

```
# app.py
import customtkinter as ctk
from ui.main_window import MainWindow

class PixelKitApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title('PixelKit')
        self.geometry('1200x750')
        self.minsize(900, 600)
        self._setup_icon()
        self.main_window = MainWindow(self)
        self.main_window.pack(fill='both', expand=True)

    def _setup_icon(self):
        import sys, os
        if sys.platform == 'win32':
            self.iconbitmap('assets/icon.ico')
        else:
            from PIL import Image, ImageTk
            img = Image.open('assets/icon.png')
            self.iconphoto(True, ImageTk.PhotoImage(img))
```

```
# ui/main_window.py
import customtkinter as ctk
from ui.sidebar import Sidebar
from ui.frames.compress_frame import CompressFrame
from ui.frames.convert_frame import ConvertFrame
# ... importar todos los frames
```

```
MODULES = {
    'compress':   ('Comprimir',        CompressFrame),
    'convert':    ('Convertir',         ConvertFrame),
    'remove_bg':  ('Quitar fondo',      RemoveBgFrame),
    'resize':     ('Redimensionar',     ResizeFrame),
    'rename':     ('Renombrar lote',    RenameFrame),
    'palette':    ('Paleta colores',    PaletteFrame),
    'watermark':  ('Watermark',         WatermarkFrame),
    'metadata':   ('Metadatos EXIF',    MetadataFrame),
    'lqip':       ('LQIP / Base64',     LQIPFrame),
    'optimizer':  ('Smart Optimizer',   OptimizerFrame),
}

class MainWindow(ctk.CTkFrame):
    def __init__(self, parent):
        super().__init__(parent, corner_radius=0)
        self.frames = {}
        self._build()

    def _build(self):
        self.sidebar = Sidebar(self, on_select=self.show_module)
        self.sidebar.pack(side='left', fill='y')

        self.content = ctk.CTkFrame(self, corner_radius=0)
        self.content.pack(side='left', fill='both', expand=True)

        for key, (label, cls) in MODULES.items():
            frame = cls(self.content)
            frame.place(relwidth=1, relheight=1)
            self.frames[key] = frame

        self.show_module('compress')  # Pantalla inicial

    def show_module(self, key):
        self.frames[key].tkraise()
        self.sidebar.set_active(key)
```

```
# ui/sidebar.py
import customtkinter as ctk

MENU = [
    ('compress',  'Comprimir'),
    ('convert',   'Convertir'),
    ('remove_bg', 'Quitar fondo'),
    ('resize',    'Redimensionar'),
    ('rename',    'Renombrar lote'),
    ('palette',   'Paleta'),
    ('watermark', 'Watermark'),
    ('metadata',  'Metadatos EXIF'),
    ('lqip',      'LQIP / Base64'),
    ('optimizer', 'Smart Optimizer'),
]

class Sidebar(ctk.CTkFrame):
    def __init__(self, parent, on_select):
        super().__init__(parent, width=200, corner_radius=0,
                         fg_color=('#1a1a2e', '#0f0f1a'))
        self.on_select = on_select
        self.buttons = {}
        self._build()
```

```
    def _build(self):
        logo = ctk.CTkLabel(self, text='PixelKit',
            font=ctk.CTkFont(size=22, weight='bold'))
        logo.pack(pady=(20, 30))

        for key, label in MENU:
            btn = ctk.CTkButton(
                self, text=label, width=170,
                fg_color='transparent',
                hover_color=('#2d2d5e', '#1a1a3e'),
                anchor='w', font=ctk.CTkFont(size=13),
                command=lambda k=key: self.on_select(k)
            )
            btn.pack(pady=3, padx=10)
            self.buttons[key] = btn

    def set_active(self, key):
        for k, btn in self.buttons.items():
            if k == key:
                btn.configure(fg_color=('#2563eb', '#1d4ed8'))
            else:
                btn.configure(fg_color='transparent')
```

5.5 Drop Zone con drag & drop

El drag & drop real en Tkinter requiere tkinterdnd2. Sin esta libreria, Tkinter no tiene soporte nativo de DnD en todos los sistemas.

```
# ui/drop_zone.py
import customtkinter as ctk
from tkinterdnd2 import DND_FILES, TkinterDnD

class DropZone(ctk.CTkFrame):
    def __init__(self, parent, on_drop, text='Arrastra imagenes aqui'):
        super().__init__(parent, height=160,
                         border_width=2, border_color='#3b82f6',
                         fg_color=('#f0f4ff', '#0d1117'),
                         corner_radius=12)
        self.on_drop = on_drop
        ctk.CTkLabel(self, text=text,
            font=ctk.CTkFont(size=14), text_color='gray').pack(expand=True)
        self.drop_target_register(DND_FILES)
        self.dnd_bind('<<Drop>>', self._handle_drop)
        self.bind('<Button-1>', self._browse)

    def _handle_drop(self, event):
        paths = self.tk.splitlist(event.data)
        self.on_drop(list(paths))

    def _browse(self, event):
        from tkinter.filedialog import askopenfilenames
        files = askopenfilenames(
            filetypes=[('Imagenes', '*.jpg *.jpeg *.png *.webp
                        *.gif *.bmp *.tiff *.avif')])
        if files:
            self.on_drop(list(files))
```

6. Dependencias y configuracion del entorno

requirements.txt

```
# requirements.txt
customtkinter>=5.2.0
Pillow>=10.0.0
rembg>=2.0.50
onnxruntime>=1.16.0
colorthief>=0.2.1
piexif>=1.1.3
pillow-avif-plugin>=1.4.0
tkinterdnd2>=0.3.0
pyinstaller>=6.0.0
```

Instalacion del entorno de desarrollo

```
# 1. Crear entorno virtual
python -m venv venv

# 2. Activar (Windows)
venv\Scripts\activate

# 2. Activar (macOS / Linux)
source venv/bin/activate

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Ejecutar la app
python main.py
```

Nota sobre tkinterdnd2 en macOS

En macOS, tkinterdnd2 requiere instalacion adicional:

brew install tcl-tk

pip install tkinterdnd2

Si falla el drag & drop en macOS, verificar que Python use el tcl-tk de Homebrew.

Alternativa: usar solo el boton de browse (click en DropZone) si DnD da problemas.

7. Plan de desarrollo por fases

Orden recomendado de implementacion. Cada fase entrega valor usable, no esperamos al final para tener algo que funcione.

Fase	Nombre	Tiempo est.	Entregable
1	**Scaffolding**	1-2 dias	Estructura de carpetas, main.py, app.py, sidebar, drop zone funcionando
2	**Core I/O**	2-3 dias	Comprimir + Convertir con UI completa, preview y export funcionando
3	**Procesamiento**	3-4 dias	Quitar fondo, Redimensionar, Recortar con previews
4	**Organizacion**	1-2 dias	Renombrar en lote con drag & drop de orden y preview de nombres
5	**Extras valor**	3-4 dias	Paleta de colores, LQIP/Base64, Smart Optimizer
6	**Pro features**	2-3 dias	Watermark batch, Metadatos EXIF, ajustes de app
7	**Build & distro**	1-2 dias	PyInstaller, icono, pruebas en Win/Mac/Linux

Regla de oro del desarrollo

No construyas todos los modulos a la vez.
Al final de la Fase 2 ya tenes una app util que comprimir y convierte.
Cada fase que completes suma valor real. Si paras en Fase 4 sigue siendo una buena app.
Esto evita el 'proyecto eterno' que nunca termina porque siempre falta algo.

8. Build y distribucion con PyInstaller

Una vez terminada la app, PyInstaller la empaqueta en un ejecutable standalone. El usuario final no necesita tener Python instalado.

Archivo de configuracion build.spec

```
# build.spec (PyInstaller)
a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],
    datas=[('assets', 'assets')],
    hiddenimports=['rembg', 'onnxruntime', 'colorthief',
                   'pillow_avif'],
    hookspath=[],
)
pyz = PYZ(a.pure)
exe = EXE(
    pyz, a.scripts, a.binaries, a.datas,
    name='PixelKit',
    debug=False,
    console=False,     # Sin consola en Windows
    icon='assets/icon.ico',
)
```

Comandos de build por sistema

```
# Windows (.exe)
pyinstaller build.spec
# Resultado: dist/PixelKit.exe

# macOS (.app)
pyinstaller build.spec
# Resultado: dist/PixelKit.app
# Para .dmg: usa create-dmg (brew install create-dmg)

# Linux (binario)
pyinstaller build.spec
# Resultado: dist/PixelKit
```

Sobre el modelo de rembg en el build

El modelo U2Net (~170MB) no se incluye en el .exe por defecto.
Al distribuir la app hay dos opciones:
A) Dejar que se descargue en el primer uso (mas liviano el instalador)
B) Empaquetar el modelo: agregar ~/.u2net/ a datas en build.spec
Para distribucion seria, la opcion B es mejor UX (funciona offline desde el dia 1).

Tamano aproximado del instalador final

- Sin modelo rembg: ~80-120 MB
- Con modelo rembg incluido: ~280-320 MB

- Recomendacion: distribuir version sin modelo, auto-descarga en primer uso de 'Quitar fondo'